

Resolve, Complete, Then Minimize: LR Parse Tables as Automata

Gerben Stavenga
stavenga@gmail.com

Abstract

Canonical LR(1) construction preserves the context needed for correct conflict resolution, but its tables often contain too many states to ship. Practical generators therefore merge states early and then prevent or repair changes caused by the merge. This paper explores the opposite order: construct the canonical automaton, resolve conflicts there, complete selected error entries, and minimize only the resulting behavior.

The key representation encodes **reduce r on a** as an ordinary transition labeled a into a state distinguished by production r . Shifts, gotos, and reductions then inhabit one labeled transition system. Subset construction removes nondeterminism in *where the machine goes*, but a resulting subset may still contain incompatible answers about *what the parser should do*: item and reduce nodes together encode a shift/reduce conflict, while two reduce nodes encode a reduce/reduce conflict. Resolution classifies this output nondeterminism before context is discarded. A conservative completion then aligns reductions across same-core states, and ordinary partition refinement computes the quotient. Completion is the classical default-reduction transformation under a per-terminal, alignment-driven selection policy. It may delay errors but preserves accepted-input action traces.

Gazelle implements this pipeline using the same generic automaton module as its lexer generator. On five tractable grammars, its resolved canonical machine is checked action by action against GNU Bison's canonical-LR output by a bisimulation over shifts, gotos, reductions, acceptance, and errors. After completion and minimization, its state counts agree with Bison's IELR mode on six grammars. The largest fixture, a normalized SQLite grammar, falls from 66,539 canonical states to 1,340; Bison IELR also has 1,340 states, while LALR has 1,319 and loses canonical distinctions. On the evaluation machine, Gazelle completed the SQLite construction and comparison in approximately six seconds; Bison's canonical mode did not finish within seven minutes. Equality of final state counts remains evidence about compactness, not a claim that the two final constructions are isomorphic or occupy the same number of bytes. The result is a simpler route to a canonical-LR-equivalent parser whose state count, on the bare grammars evaluated here, is the same as IELR.

1. Introduction

Canonical LR(1) gives a parser generator an attractive semantic baseline. Its states retain the complete one-token right context derived from the grammar, so a conflict is inherent in the grammar rather than introduced by table construction. If a generator resolves conflicts in this automaton, the resolution is applied in precisely the contexts in which each action arose.

The difficulty is size. The canonical automaton for the SQLite grammar used in this paper contains 66,539 item sets. Its IELR automaton has 1,340 states.

Size here is a question about how many states the machine has, not about how the eventual action/goto matrix is packed into bytes. Sparse-row encodings, shared defaults, and similar storage techniques remain valuable; pure encodings of a fixed matrix are orthogonal and can be applied after either machine has been constructed. Default reductions sit at the boundary: they alter error behavior to make rows cheaper to encode, and Gazelle repurposes that same kind of alteration to expose state equivalence. This paper concerns the state-count objective.

The two-orders-of-magnitude gap explains the usual construction order: build or approximate the small machine first, then perform conflict analysis on it. Unfortunately, merging LR(1) contexts can create new

conflicts, and it can also change how a real conflict is resolved. Pager's method restricts construction-time merges to compatible states [3]. IELR begins from LALR and splits states whose merges would alter canonical-LR behavior [4]. These algorithms differ in detail, but both must reason about the interaction between merging and resolution.

Gazelle reverses the dependency:

```
grammar
  -> item NFA
  -> subset construction
  -> canonical conflict resolution
  -> conservative completion
  -> partition refinement
  -> parse table
```

This order makes the semantic reference machine concrete. Resolution happens before information is discarded. Later stages may merge states only when the resolved transition system cannot distinguish them.

That description conceals a representational obstacle. A conventional LR table contains transitions for shifts and gotos, but reductions are actions attached to table cells or states. Generic DFA minimization cannot preserve information absent from the transition relation. Gazelle removes the obstacle by representing **reduce r on a** as a transition on a into a per-production accepting state. A parser action is determined by the kind of state reached: reaching an item state shifts, while reaching a reduce state r reduces by production r . If determinization reaches both kinds at once, it has made the transition deterministic without making the parser's answer deterministic: that residual choice is the conflict. Reduce states are distinguished in the initial partition during minimization.

This paper makes four contributions:

1. It gives an item-NFA encoding in which the complete LR table, including reductions, is one labeled transition system.
2. It identifies LR conflicts with output nondeterminism remaining in subset states after transition nondeterminism has been eliminated.
3. It shows that the classical default-reduction transformation, under a new alignment-driven selection policy, exposes common LALR-style merges to generic partition refinement while preserving valid-input parser behavior.
4. It checks the resolved canonical machine against Bison's canonical-LR output by bisimulation, matches Bison's IELR state counts after completion and minimization, and describes runtime precedence as an application of the representation.

The claim is deliberately narrower than global minimality. Finding the smallest conflict-free merge of canonical LR(1) states is NP-hard in general [5]. Gazelle chooses one deterministic completion of the transition system and computes the unique coarsest behavioral quotient of that completed system. Another completion could, in principle, enable a smaller sound quotient.

2. Background and problem statement

Let a grammar production be

$r: A \rightarrow X_1 \dots X_n$

and let an LR(1) item be $[A \rightarrow \alpha \cdot \beta, a]$, where a is the terminal that may follow this occurrence of A . Closure introduces productions for a nonterminal after the dot, with lookaheads selected from $\text{FIRST}(\beta a)$. Goto advances the dot over one grammar symbol. The canonical LR(1) automaton is the collection of item sets reachable under closure and goto [1, 6]. Throughout, the grammar is assumed reduced (every symbol reachable and productive) and non-cyclic (no $A \Rightarrow^+ A$) — the standard hypotheses under which no LR parser can perform an unbounded cascade of reductions without consuming input.

At runtime an LR parser keeps a stack of automaton states and holds one input token in hand. A shift follows a terminal transition and pushes its target. A reduction by $A \rightarrow \text{beta}$ removes $|\text{beta}|$ states, exposes a predecessor state, and follows its transition on A . The lookahead token is not consumed by a reduction; several reductions may occur before it is finally shifted. Two standard terms recur below: a **handle** is a right-hand-side occurrence whose replacement by its left-hand side is one step in reversing a rightmost derivation, and a **viable prefix** is a grammar-symbol sequence represented by the parser's state stack that can occur without passing the right end of a handle. Lookahead determines whether a handle may be reduced now; it does not make the occurrence a handle.

In a conventional presentation, the automaton transition relation supplies shift and goto entries while a separate action table supplies reductions. If state q contains completed item $[A \rightarrow \text{beta} \cdot, a]$, cell (q, a) receives **reduce** $A \rightarrow \text{beta}$. A shift/reduce conflict occurs if a shift is also present in that cell, and a reduce/reduce conflict occurs if two completed items install different reductions there.

2.1 Why merge-first construction is difficult

LALR merges canonical states with the same LR(0) core: the same (production, dot) pairs after lookaheads are erased. Consider the standard LR(1)-but-not- LALR grammar:

```
s = A x A | B x B | A y B | B y A;
x = T;
y = T;
```

After $A \ T$, one canonical state reduces T to x on A and to y on B . After $B \ T$, another state has the reverse mapping. Their LR(0) cores are the same, but merging them unions the lookaheads and creates reduce/reduce conflicts on both terminals. The grammar is LR(1); the conflicts are artifacts of the approximation.

The same mechanism is more subtle when the canonical automaton already has conflicts. Suppose a precedence declaration resolves a shift/reduce conflict in favor of the reduction. A merge can union the conflicted context with a same-core context in which the canonical automaton shifts unconditionally — the completed item is present there too, but without that lookahead. The merged cell holds both actions, the declaration selects the reduction, and the reduction now also fires in the context whose canonical behavior was a plain shift. Reduce/reduce resolution by rule order can migrate across contexts the same way. Thus “the conflicts were resolved” is insufficient: the merged parser may differ from the canonical parser under the same resolution policy.

IELR addresses this problem by tracking which lookahead contributions make a LALR state inadequate and splitting it until resolved behavior agrees with the canonical reference [4]. Gazelle instead constructs that reference, resolves it directly, and asks a generic minimizer which resolved states remain distinguishable.

2.2 Observable behavior

State minimization requires a precise behavioral boundary. For this paper, two parsers have the same valid-input behavior when, for every token string accepted by the canonical resolved parser, they perform the same ordered sequence of shifts and reductions and therefore construct the same parse tree. They must also reject every string rejected by that parser.

They need not report an invalid string at the same token or after the same number of reductions. This distinction permits **spurious reductions**: reductions taken only on paths that cannot lead to acceptance. A spurious reduction can delay an error, but it cannot turn an invalid string into a valid one when completion and minimization satisfy the conditions in §4.

This equivalence is appropriate for recognition and valid-input semantics. It does not preserve all observations a caller might make during a failed parse. In particular, semantic actions with external side effects may run on a doomed path before the error is reported. Applications requiring transactional behavior on invalid input must buffer or roll back such effects independently.

3. Encoding the complete table as an automaton

The construction begins with an NFA whose ordinary states are LR(1) items. For each item $[A \rightarrow \alpha \cdot X \beta, a]$, add a transition labeled X to $[A \rightarrow \alpha X \cdot \beta, a]$. If X is a nonterminal, add epsilon transitions to $[X \rightarrow \cdot \gamma, b]$ for its productions and every b in $\text{FIRST}(\beta a)$. This is the familiar item-NFA formulation of canonical LR construction [6, 7].

Gazelle adds one **reduce state** R_r for every production r — an accepting state in the lexer sense. For every completed item $[A \rightarrow \beta \cdot, a]$, it adds

```
[A -> beta ., a] --a--> R_r .
```

The reduce state is distinguished by production r : reaching it recognizes a completed right-hand side and emits the action “reduce by r .” This is the same role that a token-distinguished accepting state plays in a lexer NFA. Only R_r for the augmented start production denotes acceptance of the whole parse; every other accepting state returns control to the pushdown parser, which performs the reduction and continues.

An equivalent reading is to extend every production by its lookahead and let the dot advance one more position:

```
A -> beta . a --a--> A -> beta a .
```

All NFA edges are therefore either epsilon closure edges or ordinary dot advances. The extra completed position is shared by all lookaheads of a production because the runtime needs only the production identity after the edge has been followed.

Subset construction now produces states containing both item nodes and reduce nodes. Its transition relation has a direct runtime interpretation:

- an edge on a nonterminal whose target contains items is a goto;
- an edge on a terminal whose target contains items is a shift;
- an edge on a terminal whose target is R_r is reduction r .

For a conflict-free LR(1) grammar, every reachable target used by the parser is either an item set or one reduce node. If a subset contains incompatible kinds of answers, determinization has succeeded as an automaton construction but has not settled the parser action. That case is §4.1. Otherwise the parser loop is small:

```
push(token a):
  loop:
    target = delta(top(stack), a)
    if target is item state q:
      push q; consume a; return
    if target is a reduce state R_r for A -> beta:
      if r is the augmented start rule: accept
      pop |beta| states
      push delta(top(stack), A)
    otherwise:
      report an error
```

The stack still matters—the parser is not a finite-state recognizer—but its table is a finite labeled transition system. The distinction matters because generic subset construction and partition refinement operate on the table, not on the pushdown control surrounding it.

3.1 Correspondence with canonical LR(1)

Ignoring reduce nodes, the epsilon closure and symbol transitions are exactly those of the standard LR(1) item NFA. Subset construction therefore yields the same reachable item sets as canonical closure/goto construction. A completed item contributes an outgoing transition on exactly its LR(1) lookahead, so the transition into R_r exists exactly where the conventional action table would contain **reduce** r .

This gives a direct correspondence rather than a new parsing method:

- canonical item sets correspond to the item projection of subset states;
- shift and goto entries correspond to item-targeting edges;
- reduce entries correspond to edges into rule-distinguished reduce states.

The representation changes where actions live, not which actions the canonical construction computes.

Proposition 1 (canonical correspondence). Projecting reduce nodes out of each reachable item-bearing subset state yields the canonical LR(1) item set reached by the same symbol path, and every reachable canonical item set occurs as such a projection. Pure reduce subsets represent action outputs and project to the empty set. For every terminal and nonterminal, the encoded machine contains the corresponding shift, goto, or reduce edge if and only if the conventional canonical table contains that action. The item-bearing projection is many-to-one: a hybrid state and a pure item state can share one item set, so the encoded machine may carry more physical states than the canonical automaton has item sets (§6.1 accounts for the duplicates when counting).

The argument follows directly by induction over subset construction: epsilon closure is LR(1) closure, symbol advance is goto, and the only additional edge maps each completed item to the reduction already prescribed by its lookahead.

4. Resolve first, complete, then minimize

4.1 The nondeterminism that remains

Subset construction is deterministic: it produces at most one outgoing edge per symbol. But this removes only **transition nondeterminism**. It says where the machine goes; it does not guarantee that every NFA state collected at the destination assigns the same meaning to arriving there.

This distinction is familiar in lexer generation. If a determinized regex state contains accepting NFA states for both IDENT and KEYWORD, the DFA has one transition path but two possible token outputs. The lexer generator still needs a priority rule. Its residual problem is not graph nondeterminism but **output nondeterminism**.

The reduce-state encoding makes LR conflicts the same phenomenon. A conflict cannot appear as two competing DFA edges; it appears in their union target.

If an item advances on terminal **a** while a completed item reduces on **a**, the target of the single **a** edge contains both advanced items and a reduce node. This is a **hybrid state**: the deterministic destination says both “shift” and “reduce *r*.” If two completed items reduce on **a**, the target contains two reduce nodes and says both “reduce *r1*” and “reduce *r2*.” The state records exactly the set of actions that a conventional table would place in the conflicted cell.

```

shift item    --a-->  advanced item --+
                                     +--> subset {advanced item, R_r}
completed    --a-->  R_r -----+
                                     one edge, two parser outputs

```

Figure 1. Subset construction combines transition targets but does not choose between their parser meanings.

Thus an LR conflict is the nondeterminism subset construction cannot and should not erase: multiple semantic outputs survive in one deterministic subset. The grammar, together with one token of evidence, has not selected a unique parse action. Conflict resolution is the separate act of choosing an output.

Gazelle first reports these states, retaining their canonical item context for diagnostics and counterexample generation. Classification is then driven not by a global policy but by per-terminal declarations in the grammar. A terminal has one of five kinds:

```

plain          a conflict is an error, reported with counterexamples;
                it compiles only under a matching `expect` count
shift ELSE     shift/reduce on this terminal resolves to shift

```

```

reduce X      shift/reduce on this terminal resolves to reduce
prec OP       both actions are kept; runtime precedence decides (§5)
conflict NAME both actions are kept; the lexer decides per token (§5)

```

There is no silent default. An unannotated conflict fails generation with its counterexamples unless the grammar acknowledges the exact conflict count (`expect 3 rr;`), in which case acknowledged reduce/reduce conflicts resolve to the earlier rule; an acknowledged shift/reduce conflict resolves to shift. Resolution is data in the grammar rather than an undocumented tool default: the dangling else is the single declaration `shift ELSE`, naming the classical disambiguation where the terminal is introduced. The property that matters for this paper is timing: whatever the declarations say, classification happens on the unmerged canonical machine. After it, each reachable state has one runtime meaning — or, for `prec` and `conflict` terminals, one precisely delimited deferred question (§5).

A hybrid state can duplicate the item behavior of a pure item state elsewhere in the automaton. This is an artifact of putting alternatives into target states. Once classification discards the losing reduce nodes, ordinary minimization merges such duplicates when their outgoing behavior agrees.

4.2 Why minimization alone is insufficient

Canonical same-core states often differ only because their reductions are defined on disjoint lookahead sets. Consider states `q1` and `q2` that both reduce by `r`, with `q1` defining the reduction on `a` and `q2` defining it on `b`. Their valid continuations may be identical even though their partial transition rows differ:

	a	b
q1	R_r	error
q2	error	R_r

A minimizer correctly keeps these rows apart. Yet completing both gaps with `R_r` is harmless when `b` cannot occur at `q1` and `a` cannot occur at `q2` on a valid parse. Completion makes the latent equivalence visible:

	a	b
q1	R_r	R_r
q2	R_r	R_r

The added entries may cause reductions on invalid input. If the reduction eventually reaches an accepting continuation, however, the supposedly impossible lookahead would witness a valid continuation of the original state, contradicting the condition under which the gap was filled. Thus the new edge can postpone rejection but cannot create an accepted sentence.

4.3 Completion versus table packing

Two different compression problems are easy to conflate. **State minimization** reduces the number of states in the parser automaton; this is the subject of the paper. **Table packing** encodes the action/goto matrix of an already chosen automaton in fewer bytes. Joliat's reduced-matrix work [11] belongs to the second problem. Yacc's per-state defaults [14] and Bison's default-reduction options [12] are more closely related to this paper: they first alter the partial machine, treating selected error cells as reductions, so the altered rows admit a sparser representation. Their objective is still row storage rather than state count, but their mechanism is semantic completion rather than pure encoding.

Gazelle's pre-minimization completion has the same cell-level effect as a default reduction: an error entry is treated as a reduction already performed elsewhere in that state. The purpose and selection policy differ. A classical default chooses one reduction per state so its row can be stored compactly. Gazelle chooses per terminal across same-core states so partition refinement can identify equivalent states. The former reduces the representation of a fixed matrix; the latter changes the completed transition system presented to the minimizer and thereby reduces the state set. Recognition and accepted-input action traces cannot tell these insertions apart: in both, an error entry is replaced by a reduction that no successful parse consults. Failed-parse behavior can distinguish them. The policies may run different semantic actions on doomed paths, delay error reporting by different amounts, and change when a feedback-sensitive lexer is called.

Under either policy, every inserted entry satisfies the same two local conditions: (i) it replaces an error entry — no existing action changes — and (ii) it extends the lookahead set of a reduction the state already performs. Condition (ii) holds for Gazelle's rule because the donor sibling has the same LR(0) core, so the donated reduction's completed item is already present in the receiving state; completion never imports a reduction from elsewhere. When both techniques are modeled as completed transition relations, classical per-state defaults are a special case of these insertions, so the preservation argument of §4.4 covers their semantic effect as well. This shared model does not make their compression objectives the same. One assumption deserves note: condition (i) reads an error entry as evidence of non-viability, which is true of the canonical machine under the policies of §4.1. A resolution policy that deliberately maps *viable* lookaheads to error — yacc's `%nonassoc` — would create error cells that this argument does not cover, and such cells must be marked non-fillable, exactly as §5's virtual-twin guard already does for deferred cells. This is not hypothetical: Bison identifies `%nonassoc`, default reductions in inconsistent states, and parser-state merging as distinct causes of delayed or inaccurate syntax-error behavior, and provides lookahead correction (`parse.lac`) to address them [12].

Concretely, Gazelle groups resolved item states by LR(0) core. Within a group, for every terminal a , it inspects the reduce targets already present:

1. If every state defining a reduce transition on a agrees on the same target R_r , add that edge to siblings where a is absent.
2. If two states reduce different productions on a , leave every gap intact.
3. Never replace an existing transition.

The disagreement rule preserves the split in the LR(1)-but-not-LALR example of §2.1. There, the same terminal selects different reductions in the two states, so completion cannot make their rows equal.

The procedure is intentionally conservative. It does not search all possible ways to complete error entries, nor does it claim that its choice produces the smallest possible parser. It chooses a deterministic completion justified by same-core context and unanimous existing reductions.

The two objectives remain distinct, and Gazelle uses the shared mechanism at two stages. Alignment-driven insertion runs before minimization to reduce the number of states. Per-state defaulting runs afterwards: each minimized state elects its most frequent reduction, changes remaining error cells to that default on doomed paths, and omits matching entries from the packed row. The first use is the paper's algorithmic contribution; the second is the classical row-sparsification use of the same behavioral relaxation.

After completion, Gazelle runs iterative partition refinement. The initial partition places all item states together and places reduce states in classes distinguished by production. Refinement repeatedly splits a class when two members transition, on some label, into different current classes. At the fixed point, quotienting by the partition produces the coarsest transition- preserving merge of the completed machine. The implementation is Moore-style iterative refinement despite its historical function name `hopcroft_minimize`.

4.4 Preservation argument

Proposition 2 (resolved-behavior preservation). Given the completion rule of §4.3 and a deterministic classification of every canonical conflict, the quotient parser accepts exactly the strings accepted by the resolved canonical parser and performs the same shifts and reductions on each such string.

There are two transformations to justify.

Completion. Write a parser configuration as $(\mathbf{sigma}, \mathbf{w})$, where $\mathbf{sigma}$ is the stack of canonical states and $\mathbf{w}$ is the unread token string. Existing edges are never changed. Every configuration on an accepting canonical run therefore follows the same edge as before, and its sequence of shifts and reductions is unchanged.

For rejection, consider the first inserted edge consulted in a run. It is a reduction by $A \rightarrow \mathbf{beta}$ on lookahead $\mathbf{a}$ from a state whose canonical cell on $\mathbf{a}$ was an error. The source state nevertheless contains the completed LR(0) item $A \rightarrow \mathbf{beta} \cdot$; same-core completion has extended only its lookahead. Let the stack encode a viable prefix ending in $\mathbf{gamma} \mathbf{beta}$. If taking the inserted reduction and later accepting were possible, the accepting run would witness a rightmost derivation in which $\mathbf{a}$ can follow the corresponding occurrence of

A after **beta** is reduced. Canonical LR(1) lookahead propagation would then include **a** on that completed item and put a reduce action in the original cell, contradicting that the cell was an error. Thus an inserted reduction preserves the non-viability of the held lookahead. The repetition is licensed by an invariant worth stating: the completed LR(0) item certifies that **beta** is a handle independently of the held lookahead, so reducing it preserves the viable-prefix invariant even though **a** is not valid there. The argument therefore applies verbatim at the next inserted edge. A doomed run never reaches a shift of the held token or an accepting configuration.

The parser may pop and reduce before discovering the error, but it rejects without consuming the offending token: the correct-prefix property is preserved. The argument is local to each inserted entry and run, so choosing donors independently per terminal creates no interaction between insertions. The reduced, non-cyclic hypothesis of §2 supplies termination: every inserted edge is a locally legal reduction, and a no-input reduction cascade cannot be unbounded. It must end in a state whose row has no entry for the held token.

Quotienting. Partition refinement merges only states with the same classification and the same labeled transitions modulo the final partition. Relate a completed-machine stack $q_0 \dots q_n$ to the quotient stack $[q_0] \dots [q_n]$ pointwise. A shift preserves this relation because equivalent states have transitions on the same label into the same target class. A reduction preserves it because different rule states never merge, both stacks pop the same number of entries, and equivalent exposed states have equivalent gotos on the production's left-hand side. Induction over parser steps therefore gives the same action at each configuration and preserves acceptance in both directions.

Together, the transformations preserve accepted strings and the action trace on every accepted string. They may change the point at which a rejected string fails, as allowed by §2.2.

This remains a proof sketch of the implemented criterion, not a claim that every completion satisfying the same external semantics has been characterized. A full proof should define viability over stack configurations and prove the inserted-reduction lemma above directly from the canonical item construction. A mechanized proof or an executable equivalence checker would strengthen the result further.

4.5 Relation to NP-hard minimization

Yang proves that minimizing LR(1) state machines is NP-hard, reducing graph coloring to the question of which same-core states of a canonical machine can be merged without introducing conflicts [5]. The hard search is over merge choices on a machine that is already built: merges interact — combining one pair can enable or forbid combining others — so the minimum conflict-free merge is a global combinatorial problem. (Each such merge can equally be read as a choice of error-entry insertions, since merging states with disjoint reduce lookaheads is completing each with the other's entries; this completion view is ours, not Yang's.) Gazelle does not attack that problem. It fixes the inserted entries once, by the unanimous-reduction rule, and computes the unique coarsest quotient of the resulting completed transition system — polynomial and deterministic, but possibly finer than what a different completion, or a cleverer merge, could reach.

The word “minimize” in this paper always refers to this fixed completed machine. It does not mean globally fewest states among all sound LR tables: Yang's optimum ranges over all conflict-free merges of the canonical machine, and nothing here claims to reach it.

5. Runtime precedence as an application

Some conflicts are intentional. An expression rule such as

`expr = expr OP expr | atom`

leaves associativity and precedence to a language policy. Traditional generators resolve each conflicted table cell statically. Gazelle can instead defer selected shift/reduce choices to token data, allowing one terminal to represent operators whose precedence is known only at runtime.

All non-plain terminal declarations use the same representational device. For a modified terminal **OP**,

construction creates a virtual symbol `OP_reduce`. Shift transitions retain the real symbol; completed items use the virtual symbol on edges to reduce states:

```
item --OP-----> item target
item --OP_reduce-> reduce target
```

Because the labels differ, subset construction and minimization see an ordinary deterministic automaton. During table extraction, `shift` and `reduce` declarations select their named branch statically. The `prec` and `conflict` declarations combine the two columns into a `shift-or-reduce` entry. For `prec`, the pending construct carries precedence `p_stack` and the incoming token carries `p_token`. A higher `p_token` shifts; a lower one reduces. At equal levels, left associativity reduces and right associativity shifts. Thus, while parsing `1 + 2 * 3`, the incoming `*` shifts over the pending `+`, whereas a following `+` causes the pending `*` expression to reduce.

This is the shunting-yard decision rule embedded in an LR parser [15]: shift an incoming operator when it binds more tightly than the pending operator; reduce when it binds less tightly; and on equal levels use associativity to choose. Gazelle needs no separate operator stack or output queue. Its LR stack carries the pending precedence, and ordinary grammar reductions produce the result. At an individual conflicted cell, the `conflict` form below exposes the more general choice: instead of applying this built-in policy, the token selects the shift or reduce branch directly.

That direct choice does not make `prec` mere shorthand in the generated API. A `conflict` token carries one fixed `Shift` or `Reduce` value throughout its `push`. An incoming operator can cause several reductions before it is shifted, and the correct shunting-yard choice must be recomputed after each one against the newly exposed pending operator. In `1 + 2 * 3 * 4`, for example, the second `*` must first reduce the pending `*`, then shift over the newly exposed `+`; one fixed choice cannot do both. A semantic action could pop a separate operator stack, but it cannot revise the current token's already fixed resolution. A user could reproduce the policy with the low-level parser by resubmitting a decision after every reduction; `prec` performs that loop inside the generated parser, using the LR stack itself. This makes `prec` the specialized path for the common expression-parsing case, while `conflict` remains the escape hatch for other token-directed choices.

Completion needs one additional guard. A state may lack `OP_reduce` while having a real OP shift. Filling the virtual gap would not merely add a reduction on an invalid path; it would turn a canonical unconditional shift into a deferred choice on valid input. Gazelle therefore does not fill a virtual reduce edge into a state that has a transition on its real twin.

`conflict` terminals use the same mechanism with a different decision source: both branches survive to the table, and the lexer, rather than a precedence comparison, supplies the answer per token. Gazelle uses this explicit feedback channel for C's typedef ambiguity; the contextual decision remains in the lexer, but its effect on parsing is represented directly in the table.

The comparison below distinguishes a **table-local resolver** from an unrestricted callback. A table-local resolver chooses from the current cell using the current token and bounded metadata already maintained by the parser; it does not reread the consumed input or run a shadow parser. Without this boundary, runtime resolution is vacuous as a parser-generation result: a generator could emit a loop that asks arbitrary user code for every action, and that code could implement the canonical parser itself. Gazelle's `prec` resolver is table-local: each decision compares the incoming token's precedence with precedence already carried on the LR stack. Gazelle's `conflict` terminals are not table-local in how their one fixed per-token answer is computed: the lexer may consult arbitrary state. The boundary they respect is a different one: the invariant below guarantees that a deferred cell exists only where the canonical machine has the question. The unrestricted answer is therefore solicited only at canonical choice points, never merely because merging converted an unconditional canonical action into a question. The answer may still depend on semantic state intentionally kept outside the grammar.

Deferral makes the advantage of the construction order concrete. A deferred cell may contain only actions available in the corresponding canonical cell; otherwise runtime data could select an action that the canonical machine never offered in that context. Gazelle preserves this invariant by separating shift and reduce branches before determinization and retaining their distinct labels through minimization; the completion guard above

enforces the same invariant at the one place completion could otherwise violate it, by manufacturing a deferred question where the canonical machine had an unconditional answer.

A documentation survey found one close precedent among fourteen LR parser generators. Parglare lets productions retain dynamically filtered actions and calls a user-supplied predicate at parse time; its manual demonstrates runtime-defined operator precedence with the deterministic `Parser` API. Its worked example ranks the first distinct operator encountered lowest and each later one higher. Its callback then shifts when the incoming operator ranks higher than the pending one and reduces when it ranks lower or equal—the shunting-yard comparison, with input-defined priorities and left-associative ties. That is user-written policy rather than a built-in parser abstraction. Compared with that general callback, Gazelle carries precedence on tokens, performs the comparison directly in the generated parser on one stack, and uses its real/virtual transition representation to preserve the deferred choice through completion and minimization. The faithfulness question also separates the two: parglare's filter selects among whatever actions its merged LALR tables retain, while Gazelle guarantees that every deferred alternative was available in the corresponding canonical cell—the invariant above and the obligation below. The distinction is observable: Appendix A.4 constructs a five-production grammar on which canonical LR(1) shifts a token unconditionally in one context and has a genuine deferred question on the same token in a same-core context; parglare's merged table gives one cell to both, and either erases the question at construction or asks it in both contexts. No table-local filter can answer correctly on both sides. An unrestricted callback can recover by rereading left context or maintaining a shadow parser, but doing so moves the context reconstruction that the table omitted into user code. Each of the other thirteen surveyed generators documents only generation-time precedence in deterministic LR mode. GLR systems also offer parse-time ranking of forked parses. Appendix A records the survey and its scope.

Deferral imposes a different proof obligation on a merge-first construction. IELR's inadequacy analysis is designed to preserve the result of static conflict resolution; analogous runtime deferral would appear to require preserving the complete canonical action set and whether a conflict exists in each context. We have not implemented that extension, so this is an architectural observation rather than a complexity or impossibility claim. With Gazelle's real/virtual branch representation in place, preserving deferral through completion costs exactly that one guard. Bison's `%dprec` is superficially related but operates in GLR mode by ranking surviving parses [12]; Gazelle instead chooses one action on a single deterministic LR stack before either branch executes.

Gazelle's regression suite compares this runtime decision with a conventional fixed grammar hierarchy for all 1,093 operator sequences containing up to seven operands and three precedence levels. The two parsers produce identical abstract syntax trees in every case. This is bounded validation of the runtime mechanism, not a proof for arbitrary grammars.

The transition system preserves both branches until runtime, and partition refinement keeps apart exactly the states whose labeled choices differ. A full treatment of runtime precedence—including its interaction with lexer feedback and semantic values—is outside this paper's central claim.

6. Evaluation

Gazelle's `--yacc` mode emits an equivalent Bison grammar, allowing Bison to serve as an independent implementation reference. The evaluation uses six grammars: C11, Python, jq, SQLite, Gazelle's regular-expression grammar, and its self-hosted meta grammar. jq is mechanically normalized from its native Bison grammar; SQLite is normalized from the grammars-v4 ANTLR grammar. The sources, pinned revisions, licenses, and transformations are recorded with the fixtures. They exercise parser-table construction and are not claims of complete Gazelle front ends.

Terminal resolution modifiers (`shift`, `reduce`, `prec`, `conflict`) and upstream static-precedence declarations are stripped for the comparison, so both tools receive the same bare grammar and apply the same default conflict choices. These comparison machines are not the production Gazelle tables when a grammar uses modifiers; §6.2 reports those counts separately.

6.1 Canonical construction

The first comparison uses `bison -Dlr.type=canonical-lr` with default reductions restricted to the accepting state. Bison contributes one synthetic `$accept` state; the table reports its count minus that state.

grammar	Gazelle canonical item sets	Bison canonical – <code>\$accept</code>
C11	2,097	2,097
Python	3,298	3,298
jq	4,779	4,779
regex	69	69
meta	61	61

Conflict counts agree as well. More strongly, the automated regression exports Bison's resolved actions and compares the two reachable labeled transition systems. Starting from their initial states, every terminal and nonterminal must produce the same shift, goto, reduction, acceptance, or error; successor pairs are added to the relation recursively, and every item state in both machines must be covered. The relation is intentionally not required to be a bijection because Gazelle's hybrid encoding can contain multiple physical targets for one conventional LR state. This canonical bisimulation passes for all five grammars in the table.

SQLite has 66,539 Gazelle canonical states. GNU Bison's canonical construction did not finish within seven minutes on the evaluation machine, so no Bison canonical count or cross-tool bisimulation is reported for that fixture. It is included in the independently generated IELR comparison below.

6.2 Completed and minimized state counts

The second comparison uses Bison's IELR and LALR modes, again subtracting the synthetic accept state.

bare grammar	Gazelle final	Bison IELR – <code>\$accept</code>	Bison LALR – <code>\$accept</code>
C11	470	470	470
Python	418	418	418
jq	311	311	311
SQLite	1,340	1,340	1,319
regex	44	44	44
meta	61	61	61

Five evaluated grammars require no states beyond LALR. SQLite is the interesting case: IELR retains 21 more states than LALR to preserve canonical resolved behavior, and Gazelle retains the same number. Gazelle compresses the canonical machine by a factor of 49.7 while avoiding the merges that would alter the reference parser.

The production Gazelle grammars retain their terminal modifiers. Virtual reduce symbols and the guard of §5 can keep states separate when merging them would manufacture a static or runtime choice absent from the canonical machine:

production grammar	Gazelle states with modifiers
C11	506
Python	418
regex	44
meta	61

The 36-state C11 difference is therefore not a failure to reach the bare IELR count. It is the measured cost of semantics removed from the Bison comparison. Conversely, the equal Python, regex, and meta counts show that modifiers do not necessarily prevent the same quotient.

The bare-count equality is empirical. It does not establish that Gazelle and IELR always produce the same quotient, that the machines are isomorphic, that their encoded tables occupy the same number of bytes, or that either count is globally optimal. The present evidence supports two separate claims: the construction preserves canonical resolved behavior by its ordering and equivalence criterion, and its state-count compactness matches IELR on these bare grammars.

6.3 Implementation size and cost

The generic automaton module contains the NFA and DFA representations, subset construction, iterative partition refinement, and column equivalence used by both lexers and parsers. It is under 300 lines in the current implementation. LR-specific code constructs the item NFA, classifies conflicts, completes reduce edges, and extracts tables.

Gazelle eagerly allocates one NFA node for every (production, dot, lookahead) triple, including unreachable triples. This trades memory for simple index arithmetic; subset construction visits only reachable sets. In an optimized test build, the SQLite fixture's complete Gazelle construction and Bison IELR comparison finishes in approximately six seconds. Gazelle constructs 66,539 canonical states and minimizes them to 1,340 in that run; Bison's canonical mode was stopped after seven minutes without a result. The Bison measurements were reproduced with GNU Bison 3.8.2. These figures are engineering observations rather than a controlled performance study. A complete evaluation should report the source revision and grammar hashes, hardware, peak memory, construction time by phase, action and goto entry counts, generated-table bytes, and parse speed against other generators.

The repository's opt-in Bison regression covers C11, Python, jq, regex, meta, and a small LR(1)-but-not-LALR witness. SQLite is an extended opt-in fixture so normal development does not pay its construction cost. An IELR-only mode runs compactness checks without first requiring Bison's potentially much slower canonical construction.

6.4 Reproducing the tests

The ordinary Rust suite has no external parser-generator dependency:

```
cargo test --features codegen
```

With GNU Bison installed, the main differential regression runs canonical state and conflict counts, canonical bisimulation, and final IELR counts. A release build is recommended for jq:

```
GAZELLE_BISON_REGRESSION=1 cargo test --release --features codegen \
  bison_canonical_and_ielr_state_counts -- --nocapture
```

The SQLite stress fixture deliberately skips Bison canonical construction and runs the independently generated IELR comparison:

```
GAZELLE_BISON_REGRESSION=1 GAZELLE_BISON_EXTENDED=1 \
GAZELLE_BISON_IELR_ONLY=1 GAZELLE_BISON_GRAMMAR=sqlite \
  cargo test --release --features codegen \
  bison_canonical_and_ielr_state_counts -- --nocapture
```

GAZELLE_BISON_GRAMMAR=<name> selects any single fixture, and GAZELLE_BISON_IELR_ONLY=1 can be used independently to run only the compact state-count comparison.

7. Related work

Knuth established canonical LR(k) parsing and the regularity of viable prefixes [1]. The item-NFA account of closure and goto is established in parsing texts and lecture treatments [6, 7]. Gazelle's contribution is not subset construction itself, but placing reductions in the same labeled transition relation so that later automaton algorithms see the complete resolved table.

DeRemer's LALR construction merges states with equal LR(0) cores [2]. Pager introduced compatibility tests that permit many such merges without creating conflicts for LR grammars [3]. IELR starts from the compact LALR structure and eliminates inadequacies—places where merging would change the behavior of the canonical parser after conflict resolution [4]. Gazelle targets the same behavioral baseline from the other direction: materialize and resolve the canonical machine, then quotient its completed behavior.

Kannapinn gives the closest prior construction [9]. He builds completed canonical LR(k) machines and applies partition refinement using right-context and reduction information as Moore-style state output. He also discusses an equivalent Mealy formulation. Crucially, he observes that ordinary minimization of the bare canonical transition graph is insufficient because reduction information is absent from that graph. Per-production reduce states remove precisely this obstacle. Kannapinn assumes conflict-free LR(k) grammars; canonical conflict resolution before minimization, conservative completion, and runtime precedence are not part of that construction.

Reduction-incorporated generalized LR parsing also moves reductions into an automaton, but with a different representation and purpose. Scott and Johnstone connect reducing states to goto targets to reduce stack activity in generalized parsing [10]. Gazelle instead uses lookahead-labeled edges into rule-distinguished sink states so reductions participate in table minimization. Heilbrunner's parsing-automata treatment belongs to the broader automata-theoretic LR tradition but retains reduction information outside the bare transition relation [8].

langcc is a contemporary LR-family generator that attacks table size from the construction side [13]: a continuation-passing-style grammar transformation enlarges the accepted grammar class, and an optimized NFA construction partitions follow sets as the automaton is built, keeping it small rather than shrinking it afterwards. That route and the post-hoc one taken here — materialize, resolve, complete, quotient — are complementary rather than competing.

Table packing is a separate objective. Joliat studies reduced matrix representations for LR parser tables [11], which concerns compactly encoding a chosen table. Default reductions are the important bridge: the original yacc report documents per-state defaults [14], and Bison documents both the option and its delayed error detection today [12]. A default is not merely an encoding of the unchanged machine; it replaces error behavior by reductions so the resulting row can be stored sparsely. Gazelle reuses that behavioral relaxation with an alignment-driven selection policy, before minimization, to make states equivalent rather than merely to make individual rows sparse. It subsequently applies the classical per-state policy to the minimized machine for row storage. Thus the transformation is directly related, while the optimization objective is different.

Yang proves that minimizing LR(1) state machines is NP-hard [5]. As §4.5 explains, Gazelle computes the coarsest behavioral quotient of one deterministically completed machine; it does not search over conflict-free merges or completions, and therefore does not contradict that result — it answers a different, tractable question whose empirical adequacy §6 measures.

8. Limitations and future work

The present construction and evaluation have six important limitations.

First, the preservation argument should be made fully formal. §4.4 reduces the parser-specific content to a single lemma — inserted reductions preserve non-viability of the lookahead — with everything after it inherited from automata theory; a proof over parser configurations would make that lemma's stack assumptions explicit, and should include the formal statement that classical per-state default reductions satisfy the same insertion conditions.

Second, the canonical comparison now establishes resolved-table bisimulation, but the final comparison establishes only state-count agreement, not final machine equivalence or equal encoded size. Exporting Bison's IELR machine into a normalization that accounts for Gazelle's completion policy would permit a product-machine check of the final tables. Reporting action/goto entries and serialized bytes would test the motivating claim about tables being small enough to ship.

Third, only six grammars are measured. The two external fixtures are mechanically normalized parser grammars, and SQLite's complemented token sets are represented by explicit wildcard-class terminals. A corpus covering more LR(1)-but-not-LALR grammars, conflict policies, nullable cycles, and large generated languages would better characterize when Gazelle's fixed completion matches IELR size and when it does not.

Fourth, spurious reductions weaken immediate error detection and can execute semantic actions on invalid input. The generated parser preserves recognition and valid parses, not an identical failed-parse trace. Error recovery may also observe differences that simple recognition does not. These behaviors should be measured explicitly.

Fifth, the modifier-stripped Bison comparison isolates the minimization algorithm but does not measure the cost of Gazelle's full resolution model. The production counts in §6.2 begin to expose that cost; trace-equivalence tests over adversarial grammars should cover **shift**, **reduce**, **prec**, and **conflict** policies directly.

Finally, the current partition-refinement implementation is a simple Moore-style fixed-point algorithm, not Hopcroft's asymptotically faster algorithm despite its function name. This does not affect the quotient it computes, but the implementation and terminology should be brought into agreement.

9. Conclusion

The hard part of minimizing an LR state machine is not merely deciding which states look similar. It is preserving the meaning of conflicts while information is discarded. Merge-first constructions must predict or repair that interaction. Gazelle avoids it by changing both the order of operations and the representation on which they operate. Packing the resulting table compactly in memory is an important but orthogonal engineering step.

Encoding **reduce r on a** as an edge labeled **a** into reduce state **R_r** makes the complete parse table a labeled transition system. Subset construction produces canonical LR(1) item sets and actions together. Conflicts are reachable target states and are resolved while their canonical contexts are still intact. Conservative completion exposes safe equivalences, and generic partition refinement computes the coarsest quotient of the completed resolved machine.

The result is not a solution to globally minimal LR table construction. It is a simpler construction of a canonical-LR-equivalent parser whose state counts, on all six modifier-stripped grammars evaluated here, equal those produced by IELR. Production grammars with additional resolution semantics may retain more states, and equal state counts do not imply equal encoded bytes. More broadly, the construction illustrates a useful engineering principle: when a domain-specific object nearly fits a mature generic algorithm, changing the representation may be more effective than inventing another domain-specific algorithm.

Acknowledgments and AI assistance

This work originated during the ZeroMatter agentic coding hackathon. Generative-AI systems, including OpenAI Codex and Anthropic Claude, assisted with the implementation and review of Gazelle, the drafting and editing of this manuscript, and literature discovery and synthesis. Their use included generating and revising code and tests, proposing prose and structural edits, locating candidate prior work, and helping compare the paper's claims with available documentation and sources. The human author selected the research questions, made the final technical and editorial decisions, ran the reported experiments, reviewed the resulting code and manuscript, and takes responsibility for the accuracy and integrity of the work. The AI systems are not authors.

References

[1] D. E. Knuth, "On the translation of languages from left to right," *Information and Control* 8(6), pp. 607–639, 1965.

- [2] F. DeRemer, *Practical Translators for LR(k) Languages*, PhD thesis, MIT, 1969.
- [3] D. Pager, “A practical general method for constructing LR(k) parsers,” *Acta Informatica* 7, pp. 249–268, 1977.
- [4] J. E. Denny and B. A. Malloy, “The IELR(1) algorithm for generating minimal LR(1) parser tables for non-LR(1) grammars with conflict resolution,” *Science of Computer Programming* 75(11), pp. 943–979, 2010. <https://doi.org/10.1016/j.scico.2009.08.001>.
- [5] Wu Yang, “Minimizing LR(1) state machines is NP-hard,” arXiv:2110.00776, 2021. <https://doi.org/10.48550/arXiv.2110.00776>.
- [6] D. Grune and C. J. H. Jacobs, *Parsing Techniques: A Practical Guide*, 2nd ed., Springer, 2008.
- [7] J. Gallier, “A Survey of LR-Parsing Methods: The Graph Method for Computing Fixed Points; Computation of FIRST, FOLLOW, and LALR(1) Lookahead Sets,” University of Pennsylvania, 2008, §11.
- [8] S. Heilbrunner, “A parsing automata approach to LR theory,” *Theoretical Computer Science* 15, pp. 117–157, 1981.
- [9] S. Kannapinn, *Eine Rekonstruktion der LR-Theorie zur Elimination von Redundanz mit Anwendung auf den Bau von ELR-Parsern*, Dissertation, Technische Universität Berlin, 2001, especially ch. 4, pp. 39–54. doi:10.14279/depositonce-276.
- [10] E. Scott and A. Johnstone, “Generalized bottom up parsers with reduced stack activity,” *The Computer Journal* 48(5), pp. 565–587, 2005.
- [11] M. L. Joliat, “On the reduced matrix representation of LR(k) parser tables,” Technical Report CSRG-28, University of Toronto, 1973.
- [12] Free Software Foundation, *Bison: The Yacc-compatible Parser Generator*, version 3.8.2, manual §5.8.1 “LR Table Construction,” §5.8.2 “Default Reductions,” §5.8.3 “LAC,” and §5.9 “Generalized LR (GLR) Parsing” (%define lr.type, %define lr.default-reduction, %define parse.lac, %dprec).
- [13] J. Zimmerman, “Practical LR parser generation,” arXiv:2209.08383, 2022.
- [14] S. C. Johnson, “Yacc: Yet Another Compiler-Compiler,” Computing Science Technical Report 32, Bell Laboratories, Murray Hill, NJ, 1975.
- [15] E. W. Dijkstra, *ALGOL-60 Translation*, Stichting Mathematisch Centrum, Rekenafdeling, ALGOL Bulletin Supplement nr. 10, 1961.

Appendix A. Survey: precedence support in parser generators

This appendix records the documentation survey behind §5’s comparison. For each tool, the official manual or reference was checked (July 2026) for a mechanism that keeps both actions of a shift/reduce conflict in deterministic LR tables and decides between them at parse time from input-dependent data. Generation-time precedence declarations (%left, %right, %nonassoc, %prec, and equivalents) do not qualify: they fix each conflicted cell before the parser runs. Mechanisms that fork the parse and rank the survivors are recorded separately below; they decide at parse time but not on a single deterministic stack.

A.1 Deterministic LR generators

generator	construction	precedence mechanism	decided at
GNU Bison (LR modes)	LALR, IELR, canonical LR(1)	%left/%right/%nonassoc/%prec	generation
Berkeley Yacc	LALR	yacc declarations	generation
Menhir	LR(1), Pager’s method	%left/%right/%nonassoc	generation

generator	construction	precedence mechanism	decided at
Happy (LR mode)	LALR	<code>%left/%right/%nonassoc</code>	generation
CUP	LALR	declarations + contextual <code>%prec</code>	generation
Lemon	LALR	<code>%left/%right/%nonassoc</code>	generation
Hyacc	LR(1) (Pager PGM), LALR	yacc declarations	generation
LALRPOP	LR(1) (lane table)	none; grammar stratification	—
PLY	LALR	precedence tuple	generation
SLY	LALR	precedence attribute	generation
Racc	LALR	precedence table	generation
Rustemo (LR mode)	LR	static declarations	generation
Jison (LR modes)	SLR/LALR/LR	<code>%left/%right/%nonassoc</code>	generation
parglare Parser	modified LALR (default), SLR optional	user <code>dynamic_filter</code> over retained actions	parse time

Parglare is the one deterministic precedent found. Productions marked **dynamic** retain candidate shift and reduce actions, and a predicate called during parsing accepts or rejects each action using the parsing context. Its manual's example makes the first distinct operator encountered the lowest priority and later operators progressively higher. Its filter shifts when the incoming operator ranks higher than the pending one and reduces when it ranks lower or equal. For example, it parses `1 + 2 * 3 - 4 + 5` as `((1 + (2 * (3 - 4))) + 5)`. The example instantiates the deterministic **Parser** class. This user-supplied filter is more general than Gazelle's built-in precedence comparison, but it establishes that runtime precedence on a deterministic LR stack is not itself novel. Parglare's deterministic tables use a modified LALR construction by default, with SLR optional. The filter exposes whatever actions those tables retain, and its documentation does not relate that retained set to the canonical automaton's per-context actions, which is the faithfulness question §5 engineers. The other thirteen surveyed tools do not document a table entry deferring the choice at all.

A.2 Parse-time precedence in GLR systems

GLR tables routinely retain the shift and every applicable reduction in a conflicted cell. At runtime the parser forks over that action set, and precedence can rank the surviving alternatives. Tree-sitter's `prec.dynamic` assigns rule weights at runtime and selects the successful subtree with the greatest accumulated weight. Bison's GLR mode uses `%dprec` to prefer the surviving action with the highest declared dynamic precedence [12]. Lezer similarly permits declared ambiguities resolved during its GLR-style parse, and Happy has a GLR mode with semantic disambiguation. In each case the mechanism presupposes nondeterministic parsing; none applies to the tool's deterministic LR tables.

A.3 Runtime precedence outside LR generation

Runtime-defined operators exist in settings that are not LR table generation. Prolog's `op/3` directive alters the operator table of its reader, an operator-precedence parser. Pratt parsing and precedence climbing decide from a binding-power table at parse time, in hand-written recursive descent. Swift's `precedencegroup` and Haskell's fixity declarations feed precedence resolution phases inside those compilers. These confirm that the *language feature* is in demand. Parglare additionally shows that a general runtime action filter can provide it in a deterministic LR parser. Gazelle's narrower contribution is the token-carried table mechanism and its preservation through completion and minimization.

A.4 A witness: merging defeats table-local filtering

To make the faithfulness distinction concrete, we constructed a five-production grammar and ran it against parglare 0.21.1 (`blog/parglare_witness.py` in the repository):


```

S: 'a' M | 'b' M Z;
M: E {dynamic} | E Z {dynamic};
E: 'x';
terminals Z: 'z' {dynamic};

```

Canonical LR(1) gives the state reached by **a** **E** an unconditional shift on **z** — the completed **M** $\rightarrow$ **E** carries lookahead **{EOF}** there — while the same-core state reached by **b** **E** has a genuine shift/reduce question on **z** (**b** **x** **z** must reduce, **b** **x** **z** **z** must shift). `parglare`'s construction merges the two states and unions their lookaheads, so one cell answers for both contexts. Under the default shift preference, the question is resolved away at construction despite the `dynamic` markers, and the valid input **b** **x** **z** cannot be recovered by filtering the remaining candidate. Notably, `parglare`'s 0.21.1 manual shows its dynamic-precedence example constructed as `Parser(grammar, dynamic_filter=...)`, but the project's corresponding regression test adds `prefer_shifts=False`; without it, the documented results are not obtained. With `prefer_shifts=False` the merged cell defers — in both contexts: at the decision point, **a** **x** **z** and **b** **x** **z** present the same automaton state and the same remaining input yet require opposite actions, so a shift policy fails **b** **x** **z** and a reduce policy fails **a** **x** **z**. No filter that reads only the state, candidate action, semantic subresults, and unread input can be correct for both.

`Parglare`'s actual callback is not limited to that local information: it can inspect the full input, retain arbitrary state in `context.extra`, or run arbitrary code. The executable witness therefore also includes a history-aware filter that distinguishes the two inputs by rereading their consumed prefix and parses all four successfully. More generally, a callback could maintain a shadow canonical parser. That escape does not restore faithfulness to the LALR table; it makes user code reconstruct the context that merging erased. Counting such an unrestricted callback as a complete answer would reduce parser generation to the oracle construction excluded above. A canonical-faithful table instead keeps the contexts apart — the **a** cell a plain shift, the **b** cell the one deferred entry — which is what §5's guard preserves through completion and minimization. The witness concerns this grammar and `parglare` 0.21.1; `parglare`'s GLR parser, which forks instead of choosing, parses all four inputs without such reconstruction.

A.5 Scope

The survey records documented mechanisms in the tools and versions examined; its negative entries do not establish impossibility, and an undocumented or later mechanism would change the comparison.

The deterministic-generator rows were checked against the following official manuals and project references: GNU Bison 3.8.2 [12]; Berkeley Yacc's manual; the Menhir Reference Manual, version 20260209, §§4.1.4, 6, and 18; Happy's LR precedence documentation and GLR documentation; the CUP manual; the Lemon documentation; the Hyacc project documentation; the LALRPOP expression tutorial; the PLY documentation; the SLY documentation; the Racc reference; the Rustemo grammar documentation; the Jison documentation; and `parglare`'s dynamic-disambiguation documentation.

The comparisons outside deterministic LR use tree-sitter's grammar DSL reference, the Lezer guide, and SWI-Prolog's `op/3` reference. All web sources were accessed in July 2026.

Appendix B. Exposing LR as a library

The paper's direct contribution is the table construction. A secondary design lesson is that the capability of an LR generator is most useful when exposed as a library rather than delivered as a framework. The algorithms make subtle decisions about context, conflicts, recovery, and runtime feedback; applications should be able to use those decisions without surrendering control of their build, semantic representation, lexer interaction, or parse loop.

Here “library” means composable ownership boundaries, not merely a function callable at runtime. `Gazelle` exposes the grammar, construction, compiled table, runtime decisions, and parser loop as components. Its procedural macro is one typed embedding of those components into Rust, not a separate framework that owns the application. We regard this combination — a declarative grammar mapped to Rust enums, ordinary

trait implementations for actions, and a caller-owned loop — as a modest secondary design contribution. The appendix exhibits the design; it does not claim a comparative usability evaluation.

B.1 Construction is a library API

Table construction is available directly to programs. A tool can parse or construct a grammar, inspect its diagnostics, compile it, and choose the runtime interface it needs:

```
let grammar = gazelle::parse_grammar(source)?;
let compiled = gazelle::CompiledTable::build(&grammar)?;
let parser = gazelle::CstParser::new(compiled.table());
```

This path supports grammar analyzers, conflict tools, dynamic grammars, and alternative semantic stacks. The static interface below invokes the same construction during Rust compilation and adds typed integration.

B.2 The grammar is only a grammar

```
gazelle! {
    grammar Calc {
        start expr;
        terminals {
            NUM: _,
            prec OP: _,
            LPAREN, RPAREN,
        }
        expr = expr OP expr => binop
            | NUM => literal
            | LPAREN expr RPAREN => paren;
    }
}
```

No host-language code appears inside the grammar. Alternatives are named (`=> binop`), not implemented, so the grammar text stays a grammar: readable on its own, diffable, and reusable across independent backends. Resolution intent is part of this declaration rather than a side channel: the terminal modifiers of §4.1 (`shift`, `reduce`, `prec`, `conflict`) state per terminal how a conflicted cell is decided, a conflict on an unannotated terminal is a generation error diagnosed against canonical contexts (§4.1), and counted intentional conflicts are declared with `expect` totals in yacc style. Note also what is absent: because `prec OP` covers a whole precedence family, the grammar needs no cascade of one nonterminal per precedence level.

B.3 Embedding in Rust

`gazelle!` is a procedural macro: the constructions of §4 run inside the host compiler, so there is no separate generator invocation and no generated file to version. For each nonterminal with named alternatives, the macro emits an enum whose variants mirror those alternatives and whose fields are typed through an associated-types trait:

```
pub enum Expr<A: Types> {
    Binop(A::Expr, A::Op, A::Expr),
    Literal(A::Num),
    Paren(A::Expr),
}
```

Semantic code lives outside the macro as ordinary Rust. In particular, type errors in semantic actions generally point at the action implementation, where ordinary editor and compiler tooling applies, rather than at quoted host-language fragments inside a grammar file.

B.4 Actions as trait implementations

A backend chooses a representation per nonterminal by implementing a `Types` trait, and folds nodes by implementing `Action` per generated enum:

```
impl gazelle::ErrorType for Evaluator {
    type Error = EvalError;
}

impl calc::Types for Evaluator {
    type Num = f64;
    type Op = char;
    type Expr = f64;
}

impl gazelle::Action<calc::Expr<Self>> for Evaluator {
    fn build(&mut self, node: calc::Expr<Self>) -> Result<f64, Self::Error> {
        match node {
            calc::Expr::Binop(l, op, r) => apply(op, l, r),
            calc::Expr::Literal(n) => Ok(n),
            calc::Expr::Paren(e) => Ok(e),
        }
    }
}
```

The usual CST-versus-AST decision becomes a per-nonterminal type choice. Setting `type Expr = Box<calc::Expr<Self>>` engages a blanket implementation that materializes the full concrete tree with no action code at all; setting it to `f64`, as above, folds values during parsing and never builds a tree; setting it to `Ignore` discards the node, for validation-only parsing. The choices mix freely across nonterminals, and several backends — evaluator, tree builder, pretty-printer — can implement the same grammar.

B.5 The parse loop belongs to the caller

Generated parsers are push-based; the caller owns the loop:

```
while let Some(token) = lexer.next(&actions.ctx)? {
    // Semantic actions may update the state read by the lexer.
    parser = parser.push(token, &mut actions)?;
}
let result = parser.finish(&mut actions)?;
```

This is where the runtime side of §5 surfaces. A `prec` or `conflict` token carries its resolution datum — `Precedence::Left(6)`, or an explicit `Resolution::Shift` or `Resolution::Reduce` — and that datum plus the pending stack precedence is everything that crosses the table-local boundary. Because the loop is open, semantic actions can update state that the lexer reads on the next iteration (the classical C `typedef` feedback) without widening that boundary: whatever the lexer learns still reaches the parser only as the next token and its datum.